UNIVERSITY OF LONDON
INTERNATIONAL PROGRAMMES

BSc Computer Science and Related Subjects

CM3070 FINAL REPORT

CM3015 Machine Learning and Neural Networks
Deep Learning on a public dataset

Author: Harish Dubey
Student Number : 210201358
Date of Submission: 9 March 2024

Contents

Chapter 1: Introduction 3
Chapter 2: Literature Review 6
Chapter 3: Project Design 14
Chapter 4: Implementation 23
Chapter 5: Evaluation 37
Chapter 6: Conclusion 44
Reference 45
Appendix 48

Chapter 1: Introduction

1.1 Background

In the vast realm of agriculture, few vegetables hold the global significance that tomatoes do. Acknowledged as the world's most popular vegetable(Global Alliance for Improved Nutrition (GAIN), 2020) , the tomato not only contributes substantially to dietary habits but also serves as a linchpin in the agricultural landscape. Renowned for its rich nutritional profile, encompassing essential nutrients such as Vitamins C and A, potassium, and antioxidants (Frusciante et al., 2007), the tomato has become a culinary staple across diverse cultures. Its versatility in culinary applications and nutritional value accentuate its importance in global diets and economies. However, the tomato, despite its ubiquity, faces an unrelenting adversary—diseases inflicted by pests and pathogens alike during cultivation. This vulnerability translates into a substantial threat to global food security, with these diseases causing up to a staggering 40% loss of tomato crops, equivalent to a financial burden of $220 billion (*FAO - News Article*, 2021). The implications of such losses extend beyond economic concerns, touching on the delicate balance of food supply chains and the livelihoods of those dependent on tomato cultivation.

1.2 Project Concept Justification

The need for constant supervision and timely intervention to curb these diseases is evident. Three factors contribute to the formation of diseases in plants: the host plant itself, the presence of a favorable environment, and the pathogen responsible for causing the disease (Ahmad et al., 2023). This trio of elements forms what is commonly known as the "plant disease triangle." Typically, symptoms of plant diseases manifest from the bottom of the plant upwards, affecting various parts of the plant as the infection progresses. Once a plant is infected, many diseases have the potential to spread throughout the entire crop, posing a significant threat to agricultural productivity and crop yields. Recognizing and understanding the dynamics of the plant disease triangle is crucial for effective disease management and mitigation strategies in agriculture.

Traditionally, human experts have been the frontline defense against the proliferation of agricultural threats, relying on visual inspection and manual observation to identify and mitigate crop diseases (Mahlein, 2016). This approach often involves visually examining plants for symptoms such as discoloration, lesions, and abnormal growth patterns, followed by manual interventions such as pruning or applying chemical treatments as necessary (Ram et al., 2018). However, this method is labor-intensive, time-consuming, and subject to human error, limiting its scalability and effectiveness, especially in large-scale agricultural operations (Bock et al., 2010). Furthermore, visual inspection may not always detect diseases at early stages when they are most treatable, leading to suboptimal disease management and yield losses (Bock et al., 2010). As a

result, there is a growing need for automated and technology-driven solutions to enhance disease detection and management in agriculture.

In this context, the intersection of technology and agriculture becomes crucial. In recent years, the progression of technology has revolutionized various industries, including agriculture. With the advent of image processing and computer vision techniques(Devaraj et al., 2019), there has been a significant shift towards leveraging these technologies for crop disease detection and management. Machine learning algorithms have played a vital role in automating processes traditionally handled by human experts (Mahlein, 2016). These algorithms, particularly in the context of plant disease detection, have shown promising results by accurately identifying diseases based on visual symptoms captured through images (Rani et al., 2019). Deep learning, and specifically Convolutional Neural Networks (CNNs), have emerged as powerful tools in this domain (Chowdhury et al., 2021). CNNs excel in learning hierarchical representations of images, making them particularly suitable for tasks such as image classification and object detection (Sladojevic et al., 2016). In agriculture, CNNs have been applied to various tasks, including crop disease detection, yield prediction, and weed identification (Fuentes et al., 2017).

1.3 Objective of this paper

The project aims to develop and refine a deep learning-based classification model for the automated detection of tomato leaf diseases. Leveraging convolutional neural networks (CNNs), the project seeks to explore various model architectures and optimization techniques to improve classification accuracy and robustness. Through iterative experimentation and refinement, the goal is to create an effective and efficient solution for accurately identifying and diagnosing tomato leaf diseases, addressing the pressing need for timely disease detection in agriculture. Additionally, the paper seeks to evaluate the performance of the proposed model against traditional machine learning algorithms and pre-trained deep learning models. By comparing the results obtained from different models, the paper aims to demonstrate the effectiveness of the proposed approach in accurately identifying tomato leaf diseases. The main contributions in this paper are:

a) showcasing the process of iteration to reach the proposed model from a basic model

b) competitive performance achieved by the final model proposed while needing much less storage space.

The remainder of the paper is organized as follows: Section 2 provides a comprehensive review of the literature relevant to the classification of tomato leaf diseases. In Section 3, we detail the project methodology, dataset, and the initial model design. This section also includes a brief

introduction to traditional machine learning methods and deep convolutional neural networks. Section 4 presents the iterative experiments conducted with the initial CNN model, documenting the evolution of the model architecture and performance enhancements to attain the final proposed model. The main findings of this paper are discussed in Section 5, where we analyse the results obtained from different models and evaluate the effectiveness of the proposed approach. Finally, Section 6 offers conclusions based on the findings and suggests avenues for future research.

Word Count of chapter 1: 896

Chapter 2: Literature Review

Literature Review 1: Tomato Leaf Disease Detection Using Deep Learning Techniques

The primary objective of the paper (Ashok et al., 2020) is to address the imperative requirement for the early detection of plant leaf diseases, specifically within the burgeoning agricultural economy of India. The focus is directed towards tomato plants, proposing a holistic methodology that incorporates image processing techniques, such as segmentation and feature extraction through Discrete Wavelet Transform (DWT) and Gray-Level Co-occurrence Matrix (GLCM). Furthermore, the paper conducts experimentation and comparative analysis, pitting the proposed CNN model against other established models like AlexNet (another CNN variant) and Artificial Neural Network (ANN). The overarching goal is to furnish farmers with a dependable and accurate system for the prompt identification of tomato plant leaf diseases, thereby enabling timely implementation of preventive measures.

The paper serves as a valuable learning resource by emphasizing the superiority of CNN models (Figure 1)in the context of disease classification. Through the experimentation and comparison with alternative models such as AlexNet and ANN, it becomes evident that CNN stands out as the preferred model, exhibiting a clear advantage in accuracy and efficiency. This valuable insight serves as the cornerstone for the subsequent development of our prototype, affirming the choice of CNN as the foundational model to be employed in this report.

Table II. Comparison average performance measures of Tomato leaf disease detection

S.No	Method	Accuracy (%)
1	Deep Neural Networks (AlexNet) [17]	95.75
2	ANN technique [18]	92.94
3	Tomato leaf disease detection using CNN (**proposed method**)	98.12

Figure 1: Table for comparing average performance amongst 3 models employed obtained from (Ashok et al., 2020) paper

A significant critique pertains to the narrow focus on only four types of tomato leaf diseases (Figure 2) in the experimental section. The paper would significantly benefit from diversifying the dataset to include a broader range of diseases commonly affecting tomato plants. A more

comprehensive dataset would not only enhance the model's generalizability but also bolster its practical utility in addressing a more extensive array of real-world scenarios. This prompts me to employ a more comprehensive standard for classification types within this report, namely, a 10-class classification standard.

Table I. Average performance measures of Tomato leaf disease detection (proposed method)

S.No	Different types of tomato leafs disease	Accuracy (%)
1	Healthy	98.75
2	Phoma Rot	96.67
3	Leaf Miner	96.88
4	Target Spot	97.23

Figure 2: Table of 4 tomato leaf disease obtained from (Ashok et al., 2020)

Regrettably, the paper lacks explicit details concerning the dataset used, hardware specifications, and specific technologies employed during experimentation. While it acknowledges the use of OpenCV-Python for implementing image processing concepts, the absence of comprehensive information concerning the dataset and hardware specifications poses challenges to the reproducibility and scalability of the proposed methodology.

Literature Review 2: Disease Detection on the Leaves of the Tomato Plants by Using Deep Learning

The paper (Durmuş et al., 2017) aims to employ deep learning for the real-time detection of diseases affecting tomato plants in fields and greenhouses using mobile robotic platforms. Two deep learning architectures, namely AlexNet and SqueezeNet, are evaluated for their effectiveness in detecting diseases from images of tomato leaves. The focus is on lightweight models to enable real-time processing on mobile computers. The motivation is to contribute to precision farming by reducing the reliance on costly and environmentally harmful chemical methods for disease prevention. The study emphasizes the need for mobile computing capabilities for disease detection in the field and proposes the integration of real-time disease detection into robotic platforms for agricultural applications.

The paper demonstrates the feasibility of using deep learning, particularly lightweight models like SqueezeNet (Figure 3), for real-time disease detection on mobile robotic platforms in agricultural settings. The study highlights the advantages of a smaller model size, lower computational requirements, and faster updating speeds for mobile applications. The integration of disease detection capabilities into mobile robots is positioned as a valuable tool for precision farming, providing the potential for on-field training and adaptability to changing agricultural conditions

TABLE I Comparison between AlexNet and SqueezeNet v1.1.

Network Name	AlexNet	SqueezeNet
Accuracy on Test Set	0.9565	0.943
Model Size	227.6 Mbyte	2.9Mbyte
Inference Time	~150ms	~50ms

Figure 3: Comparison Between AlexNet and SqueezeNet obtained from (Durmuş et al., 2017) paper

The paper highlights the significance of utilizing lightweight models for real-time disease detection in agricultural settings, particularly on mobile robotic platforms. However, a notable gap exists in the evaluation process, as the study solely focuses on deep learning models, such as AlexNet and SqueezeNet, without considering the efficacy of traditional machine learning models. By overlooking traditional machine learning models, which are inherently smaller in scale compared to their deep learning counterparts, the paper misses an opportunity to provide a comprehensive understanding of the potential benefits and drawbacks of different modeling

approaches. As such, this point will be used to enhance the Final Project to include traditional models, which are known for their computational efficiency and simplicity, to determine if they could offer valuable insights into achieving an optimal balance between accuracy and resource requirements.

The PlantVillage dataset, consisting of labeled images for various crops, is utilized in the study, specifically focusing on tomato leaves. The Nvidia Jetson TX1 serves as the hardware platform for both training and testing. The Jetson TX1 features 256 CUDA cores, a quad-core ARM processor, 4GB RAM, 16 GB eMMC, and other peripherals. The deep learning frameworks Caffe, written in C++ with Python bindings, are used for training the neural networks. The selected networks, AlexNet and SqueezeNet, are trained and tested on the Nvidia Jetson TX1. In an effort to attain reproducibility and cross-referencing, a PlantVillage dataset on Kaggle was found and would also be utilized in this report.

Literature Review 3: Tomato Leaf Diseases Detection Using Deep Learning Technique

The paper (Chowdhury et al., 2021) aims to address the global issue of food insecurity caused by plant diseases, specifically focusing on tomato crops. It proposes the use of artificial intelligence, particularly Convolutional Neural Networks (CNNs), for early detection and classification of tomato diseases. The study involves experiments with different CNN architectures, such as ResNet18, MobileNet, DenseNet201, and InceptionV3, to classify tomato leaf images into binary, six-class, and ten-class categories based on health and various diseases.

The paper asserts that deep architectures, particularly DenseNet201 with 201 layers and InceptionV3 with 48 layers, demonstrate superior performance in the classification tasks. The greater the number of layers, the deeper the architecture. While the reported results exhibit notable accuracy, sensitivity, and specificity, it is imperative to scrutinize factors such as model complexity and computational demands. For instance, it is noteworthy that ResNet18 with 18 layers outperformed MobileNet with 27 layers for six-class classification(Figure 4). Shallower models outperformed deeper models in certain classifications and metric classes, challenging the notion that deeper architectures consistently yield better results.

Classification	Model	Overall	Weighted			
		Accuracy	Precision	Sensitivity	F1-score	Specificity
Binary Classification	ResNet18	98.4	98.4	98.37	98.37	95.2
	MobileNet	98.42	98.42	98.38	98.33	95.45
	DenseNet201	98.9	98.85	98.66	98.76	95.56
	Inceptionv3	**99.2**	**99.23**	**99.2**	**99.25**	**96**
Six-Class Classification	ResNet18	96.86	96.84	96.84	96.84	99.18
	MobileNet	96.74	96.76	96.74	96.74	99.25
	DenseNet201	**97.99**	**97.99**	**97.99**	**97.98**	**99.54**
	Inceptionv3	97.65	97.67	97.65	97.63	99.41
Ten-Class Classification	ResNet18	96.75	96.77	96.79	96.76	99.65
	MobileNet	97.2	97.18	97.19	97.17	99.7
	DenseNet201	**98.05**	**98.03**	**98.03**	**98.03**	**99.76**
	Inceptionv3	97.35	97.34	97.35	97.34	99.69

Figure 4: Shows the overall results obtained from the paper (Chowdhury et al., 2021)

This raises questions about the unequivocal relationship between depth and performance. The paper's claim that deep architectures outperform shallower models might not hold universally true. Moreover, this debate prompts the consideration of computational efficiency, especially in resource-constrained environments. Deeper architectures generally demand more computational power and memory, potentially posing challenges for practical deployment in scenarios with limited resources. As a response to this debate, my final year project could leverage these insights to explore the development of a fairly shallow CNN prototype model. This approach aims to strike a balance between performance and computational efficiency, addressing the trade-off between depth and practical applicability, particularly for edge devices or remote agricultural settings.

The study also uses the Plant Village tomato leaf dataset, consisting of 18,162 images categorized into ten classes. The experiments are conducted on an Intel Xeon Processor E5–2697 v4 with a NVIDIA GeForce GTX 1080 GPU. The PyTorch library and Python 3.7 are employed for implementing the CNN models. These further reinforces the Kaggle PlantVillage source as the primary dataset to be utilized.

Literature Review 4: An open access repository of images on plant health to enable the development of mobile disease diagnostics

The paper (Hughes & Salathe, 2015) aims to address the pressing need to increase global food production, which requires a 70% boost by 2050 to feed a projected population of over 9 billion. Infectious diseases currently cause a 40% reduction in potential crop yield, with some farmers experiencing up to 100% losses. Leveraging the widespread distribution of smartphones, the authors propose using machine learning and crowdsourcing, particularly through the PlantVillage platform, to develop mobile disease diagnostics. The paper introduces a dataset of over 50,000 images of healthy and infected crop leaves, curated by plant pathology experts, to facilitate the development of computer vision approaches to mitigate yield losses due to infectious diseases.

The authors established PlantVillage, an online platform inspired by crowd-driven forums, dedicated to crop health. It includes a library of open-access information on crops and diseases, providing a community-driven approach to problem-solving. For data collection, over 50,000 images of healthy and diseased plant leaves were collected from experimental research stations associated with various universities. Technicians collected leaves, photographed them in different conditions, and the images were curated for the dataset(Figure 5).

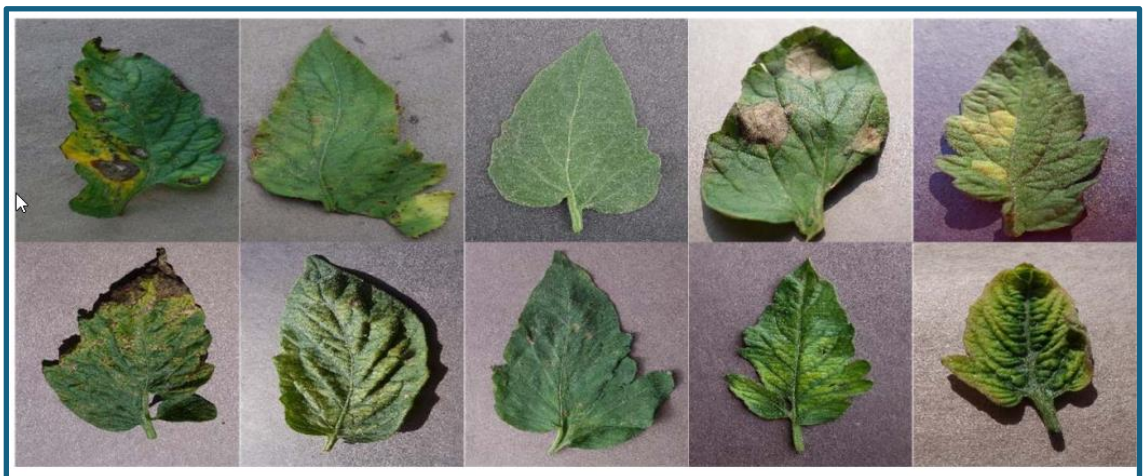

Fig. 4. Sample images from PlantVillage dataset (First row left to right labelled as bacterial spot, early blight, healthy, late blight, leaf mold; Second row left to right labelled as septoria leaf spot, spider mites, target spot, mosaic virus, yellow leaf curl virus).

Figure 5: Shows the different tomato leaves classes obtained from the repository (Hughes & Salathe, 2015)

The dataset, also accessible on Kaggle, exhibits a high level of reliability due to meticulous curation by expert plant pathologists. The diseases' identification, crucial for our project's accuracy, was determined through standard phenotyping approaches, both in controlled experiments and sentinel plots. This thorough validation process ensures that the dataset comprises only leaves expertly identified by professionals, bolstering its credibility and utility. Drawing upon this reliability, our project aims to leverage the tomato focused information for our specific objectives, specifically for 10 class classification.

Word Count of Chapter 2: 1386

Chapter 3: Project Design

In this chapter, I explain in detail the methodology implemented for the detection of diseases in tomato leaves to achieve the project objective mentioned in section 1.3. In general, the model implemented would take in tomato leaf images as input and after processing, outputs the following information:

1) Whether the leaf is healthy, if not, the type of disease (true value)
2) the predicted value outputted by the model
3) the prediction percentage

Figure 6 shows the complete methodology that will be applied for the process of detection and classification of diseases in tomato leaves. The methodology effectively contains 5 main parts:

a) Dataset Acquisition and Balancing
b) Dataset distribution
c) Creation and implementation of models
d) Iterative Development of Proposed model
e) Evaluation of Results

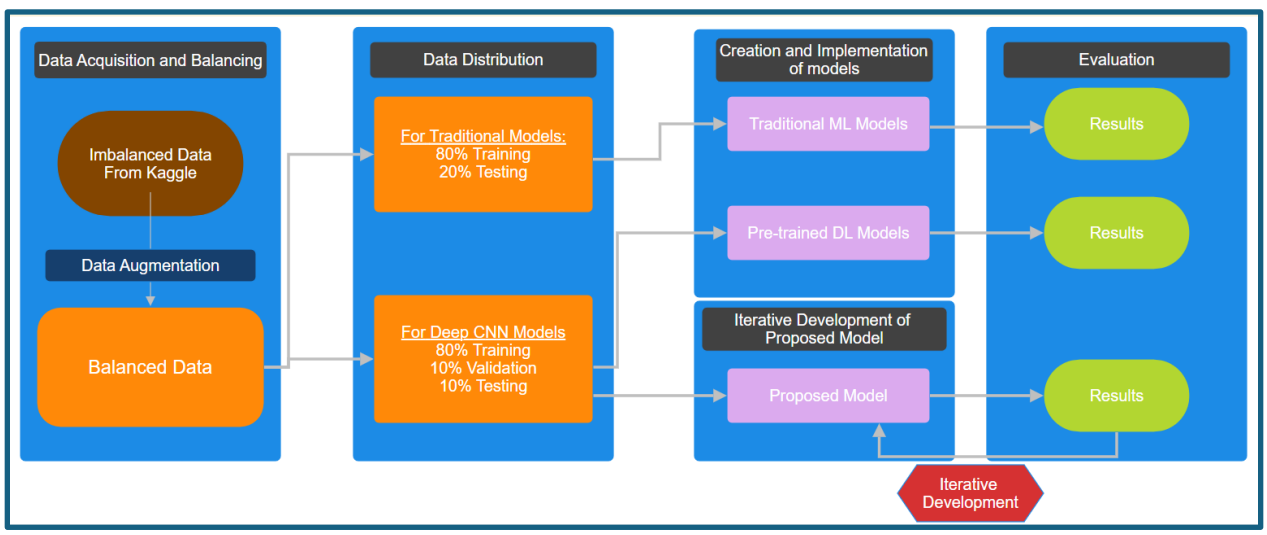

Figure 6: showcases overall methodology of the project

3.1 : Dataset Acquisition and Balancing

Kaggle, as a platform for data science and machine learning competitions, was explored as per the template requirements for dataset acquisition. The dataset that will be utilized is the PlantVillage[1] dataset uploaded by Arjun Tejaswi. This had included around 21,000 leaf images of 3 different crops, namely potato, pepper bell and tomato. Healthy and diseased classes together

[1] https://www.kaggle.com/datasets/arjuntejaswi/plant-village/data

constituted a total of 15 classes. As our project concerns tomatoes, the potato and pepper bell images were discarded, and data exploration was carried out with the remaining images. This resulted in the manual removal of a corrupted file and more importantly, identifying clear class imbalances within the dataset. The image samples within the folders of each class ranged from 373 images all the way up to 3203 images.

To mitigate the drastic class imbalances, data augmentation has been carried out to ensure that each class had around 3000 images (Figure 7). Data pre-processing techniques such as rotation, width and height shifting, shear, zoom, horizontal flipping, and fill mode adjustment were employed using the ImageDataGenerator from the Keras library. These augmentation parameters were selected to introduce realistic variations during the creation of new images, thereby not only balancing the data but enhancing the model's ability to generalize to unseen data and improving its robustness against variations in input images.

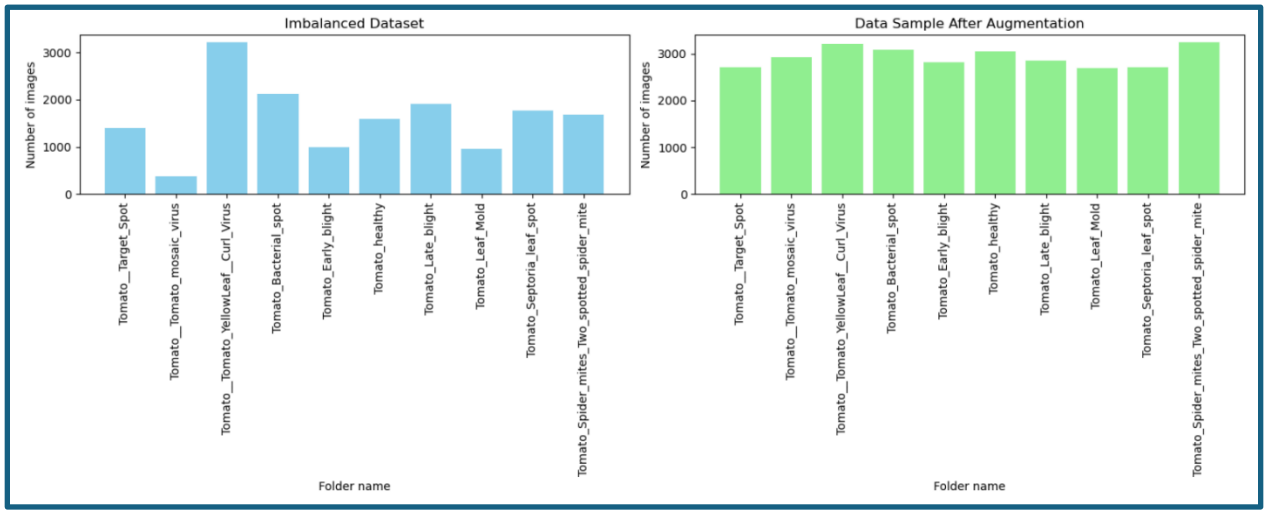

Figure 7 shows the dataset before and after Data Augmentation

3.2: Dataset Distribution

For traditional ML models, a common practice is to split the data into training and test sets using an 8:2 ratio, where 80% of the data is used for training and the remaining 20% for testing. This ratio ensures that a sufficient amount of data is available for model training while still retaining a sizable portion for evaluating the model's performance on unseen data. On the other hand, for deep learning models, a more common practice is to split the data into training, validation, and test sets using an 8:1:1 ratio, where 80% of the data is allocated for training, 10% for validation, and 10% for testing. This distribution allows for sufficient data for training, tuning hyperparameters using the validation set, and evaluating the final model performance on the test set.

These data distribution strategies will be implemented for this project.

3.3 Models to be implemented

In this subsection, a brief overview will be conducted for the traditional ML algorithms along with the pre-trained CNN Models intended for implementation.

3.3.1: Traditional Machine Learning Technologies:

The following methods used a feature set which is in the form of an input vector. The feature values are calculated based on the images provided, downscaling them to 32x32 due to computational limitations and normalizing the raw pixel values, subsequently turning the 3-Dimensional images into 1-Dimensional form using NumPy's array function.

3.3.1.1: Decision Trees

Decision tree learning employs a divide-and-conquer strategy, conducting a greedy search to identify optimal split points within the tree. This process of splitting is repeated in a top-down, recursive manner until all or the majority of records are classified under specific class labels. The effectiveness of these splits will be evaluated using Gini impurity, which quantifies the degree of impurity in a dataset, with lower values indicating better attribute splits. This (Pedregosa et al., 2011) is modelled by the following:

If a target is a classification outcome taking on values 0,1,…,K-1, for node m, let

$$p_{mk} = \frac{1}{n_m} \sum_{y \in Q_m} I(y = k)$$

be the proportion of class k observations in node m. If m is a terminal node, probability prediction for this region is set to p_{mk}. The Gini impurity is measured via:

$$H(Q_m) = \sum_k p_{mk}(1 - p_{mk})$$

3.3.1.2: K-Nearest Neighbors (K-NN)

In KNN, each data point is considered a potential neighbor to other data points in the dataset. The algorithm classifies or predicts the label of a data point by considering the labels of its nearest neighbors. The Minkowski distance metric will be used to calculate the distance between a selected point and its neighbors, which is given by the following formula(Klein, 2024):

$$d(x, y) = (\sum_{i=1}^{n} | x_i - y_i|^p)^{\frac{1}{p}}$$

where p = 1 achieves the Manhattan Distance and p =2 achieves the Euclidean distance.

3.3.1.3: Random Forest

Random Forest is a versatile ensemble learning algorithm, belonging to the family of tree-based models, and is known for its robustness and high predictive accuracy. Random Forest (Figure 8) is an ensemble of decision trees, where each tree in the forest is constructed independently and then aggregated to make predictions.

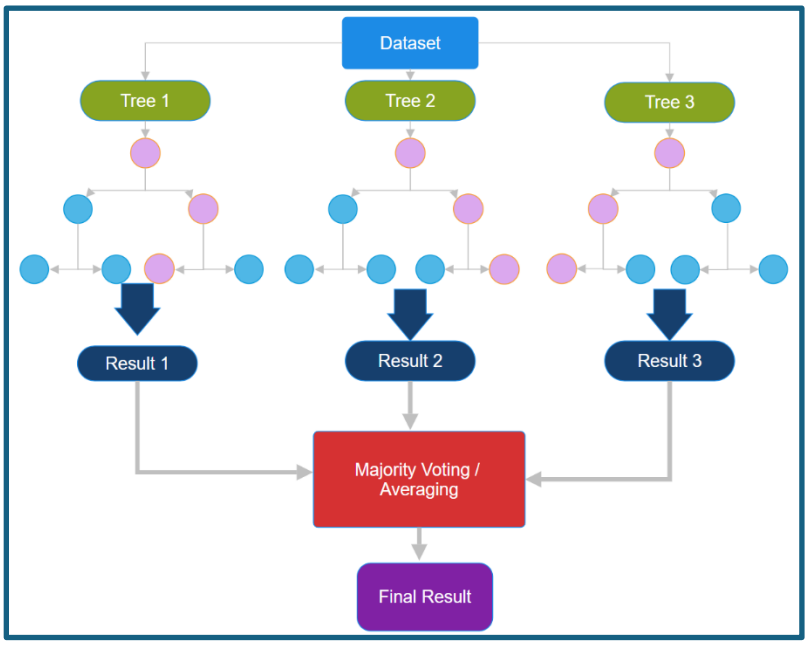

Figure 8 shows the overview of the Random Forest algorithm

3.3.1.4: Gaussian Naïve Bayes

Naïve Bayes is a probabilistic model that calculates the probability of a data point belonging to each class based on the observed features. It assigns the class label with the highest probability as the predicted class for the data point. Naïve Bayes is based on Bayes' theorem, which describes the probability of a hypothesis given the evidence and goes on to make the 'naïve' assumption that the features are conditionally independent given the class label.

This can be modelled(Pedregosa et al., 2011) by the following:

$$P(x_i \mid y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$$

Where the parameters σ_y and μ_y are estimated using maximum likelihood.

3.3.2: Pre-trained CNN Models

These following pre-trained models are already designed CNN models that were trained on the ImageNet dataset and were available from Keras Applications module. These will be implemented and run for 10 epochs to allow them to generalize on our dataset – which would be converted into a TensorFlow dataset beforehand. To further optimize the effects of transfer learning to our context, the models will be fitted according to Table 1 for the classification of 10 types of tomato

leaves. For standardization and limited computational capacity, all deep models will receive an input image of size 128x128 but will be implemented with their respective pre-processing techniques[2]. Early stopping for model training is also implemented, ensuring that the training process stops if the validation loss stops improving, with a patience of 5 epochs, and restores the best weights achieved during training.

Layers	Parameters
Lambda (Appropriate Pre-processing)	
Pre-trained Model	Include_top = 'False', input_shape : (128,128,3)
GlobalAveragePooling2D	
Dense	Units: 1024, activation = 'relu'
Dropout	0.5
Dense	Units: 10, activation = 'softmax'

Table 1 shows the general implementation of the pre-trained models

3.3.2.1: MobileNet

MobileNet(Howard et al., 2017), a pioneering CNN architecture optimized for mobile and embedded applications. Leveraging depth-wise separable convolutions, it achieves lightweight yet deep networks. With two key hyperparameters, width multiplier and resolution multiplier, MobileNet efficiently balances accuracy and storage requirements. Tailored for mobile devices with limited storage, MobileNet comprises 27 convolution layers, followed by average pooling and fully connected layers, culminating in a softmax output layer.

3.3.2.2: MobileNetV2

MobileNetV2(Sandler et al., 2019) employs an inverted residual structure, featuring thin bottleneck layers at the input and output of residual blocks. Utilizing lightweight depthwise convolutions in the intermediate expansion layer enhances feature filtering efficiency. Removing non-linearities in narrow layers preserves representational power, enhancing performance. MobileNetV2's design allows for the decoupling of input/output domains from transformation expressiveness, facilitating further analysis.

3.3.2.3: EfficientNet B1

EfficientNet B1(Tan & Le, 2020) employs a groundbreaking compound scaling method that uniformly scales all dimensions of depth, width, and resolution. This approach utilizes a simple yet highly effective compound coefficient to balance model complexity and computational

[2] With the exception of EfficientB1Net, as it has input pre-processing as part of its model.

resources. The architecture features a streamlined design comprising multiple layers, including convolutional and pooling layers, followed by fully connected layers and a softmax output layer.

3.3.2.4: DenseNet201

DenseNet201 (Huang et al., 2018) embraces the concept of dense connections, connecting each layer to every other layer in a feed-forward manner. This results in a substantial increase in the number of connections compared to traditional convolutional networks, promoting stronger feature propagation and reuse. With L(L+1)/2 direct connections, DenseNet201 effectively alleviates the vanishing-gradient problem and reduces the number of parameters while maintaining high performance.

3.3.2.5: InceptionV3

InceptionV3 (Szegedy et al., 2015) incorporates several improvements such as Label Smoothing, factorized 7 x 7 convolutions, and an auxiliary classifier to propagate label information to deeper layers. Additionally, batch normalization is applied to layers in the side head, further optimizing network performance. These enhancements contribute to InceptionV3's superior accuracy and efficiency in image classification tasks.

3.3.2.6: Xception

Xception (Chollet, 2017b) is a novel convolutional neural network (CNN) architecture inspired by the Inception family. Xception interprets Inception modules as an intermediate step between regular convolution and depthwise separable convolution operations. In Xception, Inception modules are replaced with depthwise separable convolutions, resulting in a more efficient use of model parameters.

3.3.2.7: ResNet50

ResNet50(He et al., 2015) employs residual connections, a key innovation enabling the learning of residual functions that map input to output. These connections empower ResNet50 to achieve unprecedented depth without encountering vanishing gradient issues. ResNet50 is a 50-layer convolutional neural network, composed of 48 convolutional layers, one Max Pool layer, and one average pool layer.

3.3.2.8: VGG19

VGG19 (Simonyan & Zisserman, 2015) features an architecture with very small (3x3) convolution filters, resulting in significant performance improvements compared to prior configurations. Its convolutional layers utilize a minimal receptive field of 3x3, complemented by 1x1 convolution filters and ReLU activation functions. With fixed convolution stride and fully connected layers, VGG19 achieves remarkable accuracy in image classification tasks.

3.4 Architecture of Initial Proposed Model

Before inputting the images into CNN model, they will be inputted into the following 2 sequential pre-processing layers (Figure 9). These layers perform normalization by dividing pixel values by 255 and consequently incorporate random horizontal and vertical flipping along with a random rotation with a maximum angle of 0.2 radians. These techniques aim to enhance the model's robustness by generating variations of input images during training.

```
resize_and_rescale=tf.keras.Sequential([
    #tf.keras.layers.experimental.preprocessing.Resizing(image_size, image_size), #images are 128x128
    tf.keras.layers.experimental.preprocessing.Rescaling(1.0/255)
])
# Data augmentation
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.experimental.preprocessing.RandomFlip("horizontal_and_vertical"),
    tf.keras.layers.experimental.preprocessing.RandomRotation(0.2)
])
```

Figure 9 shows further preprocessing steps like normalization and augmentation for robustness

In line with verifying the claims of the 3rd literature review, the initial CNN model, named model_V1, is designed to be simple with only two convolutional layers. Table 2 and 3 showcase the architecture and training parameters used.

Layers	Parameters
resize_and_rescale	
data_augmentation	
Conv2D	Filters: 64, kernel size: (3,3), activation = 'relu' input shape: (32, 128, 128, 3)
MaxPool2D	Pool size: (2,2)
Conv2D	Filters: 64, kernel size: (3,3), activation = 'relu'
MaxPool2D	Pool size: (2,2)
Flatten	
Dense	Units: 10, activation = 'softmax'

Table 2: Information on the layers structure on initial model

The initial proposed convolutional neural network (CNN) model consists of two convolutional layers followed by max pooling layers to reduce spatial dimensions. The first convolutional layer comprises 64 filters with a 3x3 kernel size and ReLU activation, while the second convolutional layer also consists of 64 filters with a 3x3 kernel size and ReLU activation. Subsequently, max pooling layers with a 2x2 pool size are applied after each convolutional layer to further decrease the spatial dimensions. The output from the convolutional layers is then flattened into a 1-dimensional tensor using a flatten layer. Finally, the flattened tensor is connected to a fully

connected dense layer with softmax activation, which produces class probabilities for the given input image.

Parameters	Value
Optimization algorithm	Adam
Loss Function	Sparse categorical cross-entropy
Batch Size	32
Number of epochs	30
Steps per epoch	732
Validation steps	91
Activation function for Conv Layers	ReLU

Table 3: Training Parameters of initial model

3.5 Technologies Used

The following technologies were utilized for this project.

- TensorFlow (with Keras API): TensorFlow[3], an open-source machine learning framework, leverages its Keras API for streamlined development, training, and evaluation of deep learning models.

- OpenCV: A robust image processing tool, handles tasks like image reading, resizing, and preprocessing to ensure optimal input data quality.

- Jupyter Notebook[4]: An interactive platform for iterative model development, allowing step-by-step execution and immediate feedback.

- Scikit-learn: This Python library offers efficient tools for implementation of traditional models, generating test-train splits, classification reports and evaluation metrics.

- Matplotlib and Seaborn: These Python libraries enable versatile data visualization, creating graphs and plots to visualize dataset characteristics and model performance metrics.

- Pandas and NumPy: These libraries provide powerful data manipulation and analysis tools, facilitating efficient handling and analysis of dataset features.

[3] TensorFlow version 2.8.3 was utilized.

[4] Due to computational limitations, Google Collab with their cloud GPUs and TPUs were attempted for employment, but the duration of execution was surprisingly much longer, and hence the project was downscaled in terms of Epochs, Image sizes and other aspects into a local Jupyter Notebook.

3.6 Evaluation Metrics

In the context of the tomato plant health classification project, the following evaluation metrics will be employed:

<u>Main metric:</u>

Accuracy is a fundamental metric that measures the overall correctness of the model by calculating the ratio of correctly predicted instances to the total instances as shown below:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP is True Positive, TN is True Negative, FP is False Positive, and FN is False Negative.

This metric also allows me to cross-refer to the literature reviews that utilized the PlantVillage dataset for a relatively fair comparison of the different models.

<u>Supplementary metrics:</u>

Other popular metrics employed are Precision, Recall and F1 (Harmonic Mean) , which are modelled by the equations below.

1) $Precision = \frac{TP}{TP+FP}$
2) $Recall = \frac{TP}{TP+FN}$
3) $F1 = \frac{2*Precision*Recall}{Precision+Recall}$

Furthermore, for visual illustration of the detailed breakdown of the model's predictions, a confusion matrix will also be employed.

Additionally, the evaluation will include:

- Model Size: Indicates the storage space required by the model
- Model Depth: Refers to the number of hidden layers in the model architecture, serving as a proxy for complexity

<u>Word Count of Chapter 3: 1999</u>

Chapter 4: Implementation

This section describes the experiments performed onto the balanced tomato leaf dataset, that now has 29,294 images, with the initial proposed model, and subsequent iterated models. As all the models are not pre-trained, all experimental models will undergo training for 30 epochs to learn the features of our dataset and be tested on around 300 images of each class. Just like for the pre-trained models, early stopping during model training is also implemented, ensuring that the training process stops if the validation loss stops improving, with a patience of 5 epochs, and restores the best weights achieved during training.

4.1 Performance of model_V1:

The figure below shows the training and validation plots that the model achieved during training.

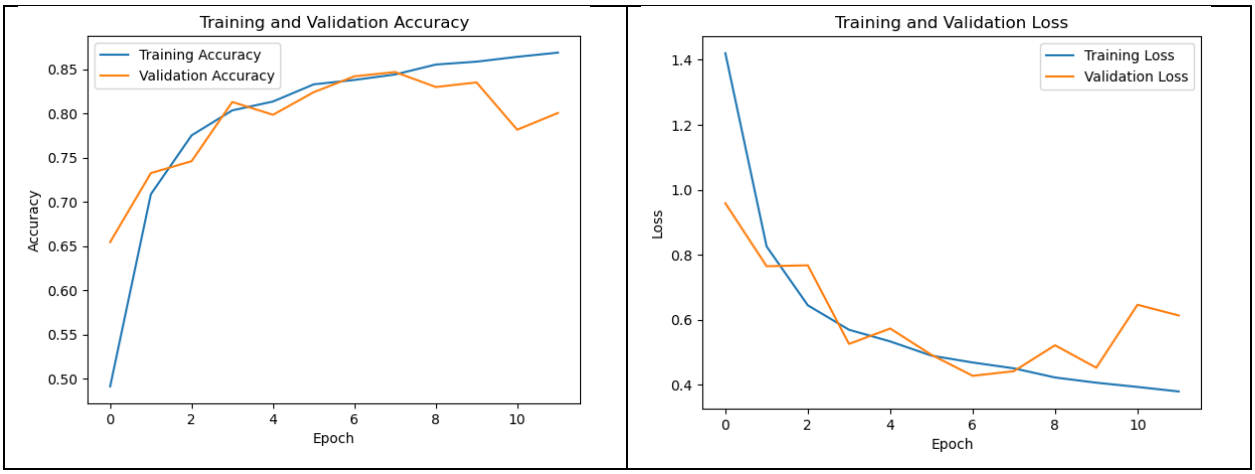

Figure 10 shows the training and validation plots for model_V1

The significance of these plots lies in the early stopping behavior observed during training. As depicted in the plot, the model training was stopped at Epoch 12 out of 30, indicating that the model's performance did not improve significantly beyond this point. This behavior aligns with the early stopping mechanism implemented, which aims to prevent overfitting and improve the generalization of the model by halting the training process when the validation loss ceases to improve.

Accuracy	0.846
Precision	0.852
Recall	0.846
F1	0.845

Table 4 shows the Evaluation metrics achieved by Model_V1 on the test dataset

Table 4 depicts the accuracy, precision, recall, and F1 score on the test set, with results hovering around 84.6% to 85%. These metrics indicate decent performance for the initial proposed model (model_V1) with a shallow layer architecture.

4.2.1 Creation and Implementation of second model:

To achieve better results for the next model, an additional convolutional layer was added to capture the hierarchical features at different levels of abstraction. As the depth of the network increases by 1 more layer, it becomes more adept at learning intricate patterns and representations present in the input images. The inclusion of an additional MaxPooling2D layer afterwards results in the model progressively reducing spatial dimensions of the feature maps. This aids in discarding irrelevant spatial information while retaining the essential features, and also helps in achieving translation invariance, where the model becomes robust to slight translations or shifts in the input images, improving the model's ability to generalize.

Furthermore, a Dense layer was added after the Flatten layer, which facilitates the fusion and abstraction of features extracted by the preceding convolutional layers. It combines the spatial information from the flattened feature maps into a more abstract representation, potentially capturing higher-level features relevant for classification, enhancing representation learning. With the increased model capacity, the addition of a Dropout layer with a rate of 0.5 after the Dense layer introduces a form of regularization, mitigating overfitting by randomly dropping out neurons during training. By introducing randomness and forcing the network to learn redundant representations, Dropout helps the model continue training beyond the point where it would typically start overfitting. This can potentially lead to better generalization and performance on unseen data.

As a result of iterative experimentation, the next best model architecture is shown in the following table:

Layers	Parameters
resize_and_rescale	
data_augmentation	
Conv2D	Filters: 32, kernel size: (3,3), activation = 'relu' input shape: (32, 128, 128, 3)
MaxPool2D	Pool size: (2,2)
Conv2D	Filters: 64, kernel size: (3,3), activation = 'relu'
MaxPool2D	Pool size: (2,2)
Conv2D	Filters: 64, kernel size: (3,3), activation = 'relu'
MaxPool2D	Pool size: (2,2)
Flatten	
Dense	Units: 64, activation = 'relu'
Dropout	0.5
Dense	Units: 64, activation = 'softmax'

Table 5: Information on the layers structure on model_V25

It is worth noting that manual hyperparameter tuning was employed instead of using Python libraries like Hyperas for automatic hyperparameter optimization. This decision was made to address the issue of validation-set overfitting, as updating hyperparameters based on a signal computed using validation data can lead to overfitting, due to the model essentially being trained on the validation data. After experimenting with different convolution units and dropouts, model_V25[5] was identified as the best model architecture.

4.2.2 Performance of model_V25

Figure 11 demonstrates the performance of model_V25 in the training and validation stages for identification and classification of tomato leaf diseases. It ran for the entire 30 epochs as expected and the results achieved a maximum validation accuracy of 96.03%.

[5] Model_V25 was meant to denote model version 2.5, not model version 25.

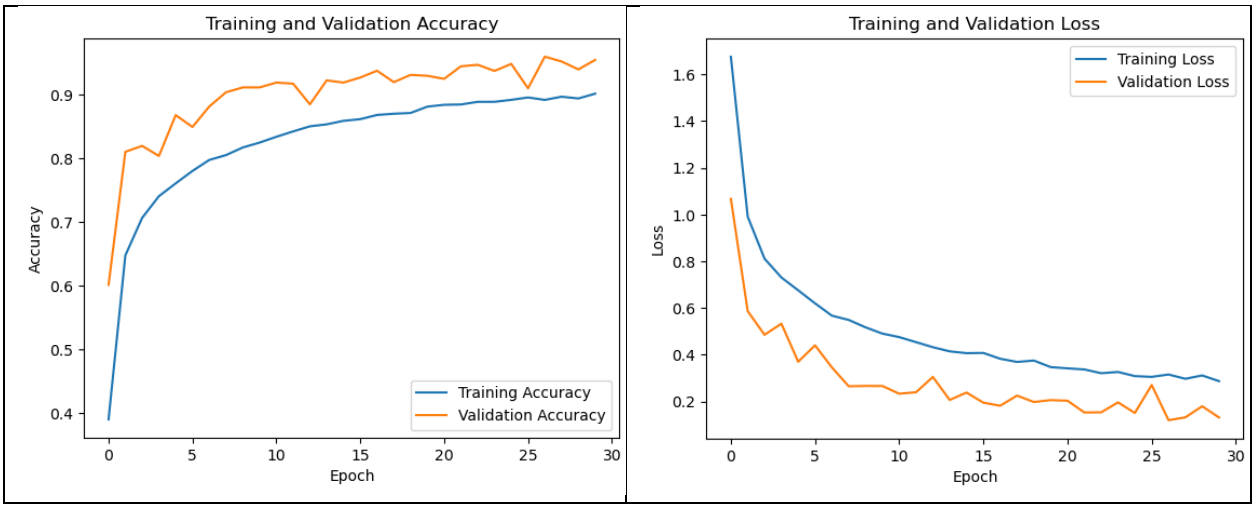

Figure 11 showing the training and validation plots for model_V25

The validation accuracy consistently outperforms the training accuracy throughout the training process. This indicates that the model is generalizing well to unseen data, as the validation set accuracy is higher than the training set accuracy. Similarly, the validation loss consistently remains lower than the training loss, suggesting that the model is effectively minimizing its loss function during training.

Accuracy	0.951
Precision	0.952
Recall	0.951
F1	0.951

Table 6 depicts the results of model_V25 on the test dataset

Table 6 presents accuracy, precision, recall, and F1 scores for model_V25 on the test dataset, showing high performance metrics, with values around 95.1-95.2% across all metrics. These high scores indicate that the model is effective in correctly identifying and classifying tomato leaf diseases.

Overall, the classification report (Table 7) of model_V25 reflects its strong performance in accurately identifying and classifying various types of tomato leaf diseases, as evidenced by high precision, recall, and F1 scores across all classes, along with consistent support values.

	Precision	*Recall*	*F1*	*Support*
0	0.98	0.93	0.95	318
1	0.92	0.95	0.93	292
2	0.95	0.93	0.94	264
3	0.98	0.94	0.96	242
4	0.91	0.95	0.93	308
5	0.98	0.87	0.93	334
6	0.92	0.94	0.93	265
7	0.97	0.99	0.98	330
8	0.95	1.00	0.97	300
9	0.96	1.00	0.98	323

Table 7: Classification Report of model_V25 on test dataset[6]

4.3.1 Creation and Implementation of third model

To attempt to achieve better results, the architecture of the third model was modified to include 2 additional sets of convolutional and MaxPooling2D layers to be able to extract a richer set of features while discarding irrelevant information, along the same line of reasoning mentioned above. Furthermore, the model was modified to incorporate these layers with increasing filter sizes applied towards the input. This adjustment was made based on the understanding that deep learning models tend to learn better representations of features closer to the input layers. By gradually decreasing the number of filters from 256 in the first convolutional layer to 32 in the subsequent layers, the model is encouraged to extract and refine intricate patterns and representations present in the input images.

[6] From 0-9, in order, the numbers refer to 'Tomato Bacterial spot', 'Tomato Early blight', 'Tomato Late blight', 'Tomato Leaf Mold', 'Tomato Septoria leaf spot', 'Tomato Spider mites Two spotted spider mite', 'Tomato Target Spot', 'Tomato Yellow Leaf Curl Virus', 'Tomato mosaic virus', 'Tomato healthy'

As a result, the third model architecture is shown in the table below.

Layers	Parameters
resize_and_rescale	
data_augmentation	
Conv2D	Filters: 256, kernel size: (3,3), activation = 'relu' input shape: (32, 128, 128, 3)
MaxPool2D	Pool size: (2,2)
Conv2D	Filters: 128, kernel size: (3,3), activation = 'relu'
MaxPool2D	Pool size: (2,2)
Conv2D	Filters: 64, kernel size: (3,3), activation = 'relu'
MaxPool2D	Pool size: (2,2)
Conv2D	Filters: 32, kernel size: (3,3), activation = 'relu'
MaxPool2D	Pool size: (2,2)
Conv2D	Filters: 32, kernel size: (3,3), activation = 'relu'
MaxPool2D	Pool size: (2,2)
Flatten	
Dense	Units: 64, activation = 'relu'
Dropout	0.5
Dense	Units: 64, activation = 'softmax'

Table 8: Information on the layers structure on model_V3_1

4.3.2 Performance of third model

Figure 12 demonstrates the performance of model_V3_1 in the training and validation stages for identification and classification of tomato leaf diseases. It ran for the entire 30 epochs as expected and the results achieved a maximum validation accuracy of 95.37%, slightly lower than the second model.

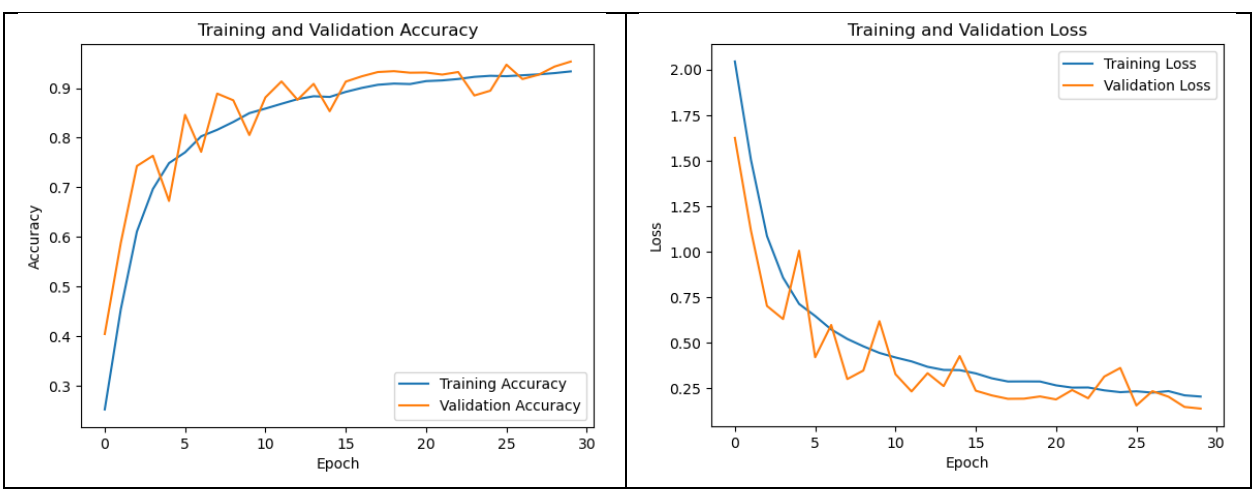

Figure 12 showing the training and validation plots for model_V3_1

Accuracy	0.948
Precision	0.948
Recall	0.948
F1	0.947

Table 9 depicts the results of model_V3_1 on the test dataset

The table above (Table 9) presenting accuracy, precision, recall, and F1 scores for model_V3_1 on the test dataset shows relatively high-performance metrics, with values around 94.7%-94.8% across all metrics. However, these metrics are slightly lower than the ones procured by model_V25. It is worth noting that during the hyperparameter tuning, another variant of this model which has 64 filter sizes for all its convolutional layers halted its training in the 24th epoch out of 30 and averaged marginally better at 94.9%. However, the architecture and results of model_V3_1 is discussed here instead as its implementation will be later revisited in section 4.5.

	Precision	*Recall*	*F1*	*Support*
0	0.94	0.94	0.94	318
1	0.93	0.95	0.94	292
2	0.91	0.95	0.93	264
3	0.94	0.99	0.96	242
4	0.96	0.87	0.91	308
5	0.94	0.93	0.93	334
6	0.94	0.89	0.91	265
7	0.96	0.98	0.97	330
8	0.96	1.00	0.98	300
9	0.98	0.98	0.98	323

Table 10: Classification Report of model_V3_1 on test dataset

Overall, the classification report (Table 10) of model_V3_1 also reflects its strong performance in accurately identifying and classifying various types of tomato leaf diseases. While model_V3_1 exhibits strong performance across various metrics, achieving high precision, recall, and F1 scores for each class on the test dataset, when pitted against model_V25, it may outperform in certain cases while potentially underperforming in others.

4.4.1 Utilization of Depthwise Separable Convolution on model_V25

In an attempt to make the best proposed model (model_V25) so far, achieve better results while making it more lightweight at the same time, depthwise separable convolution (Figure 13) was implemented.

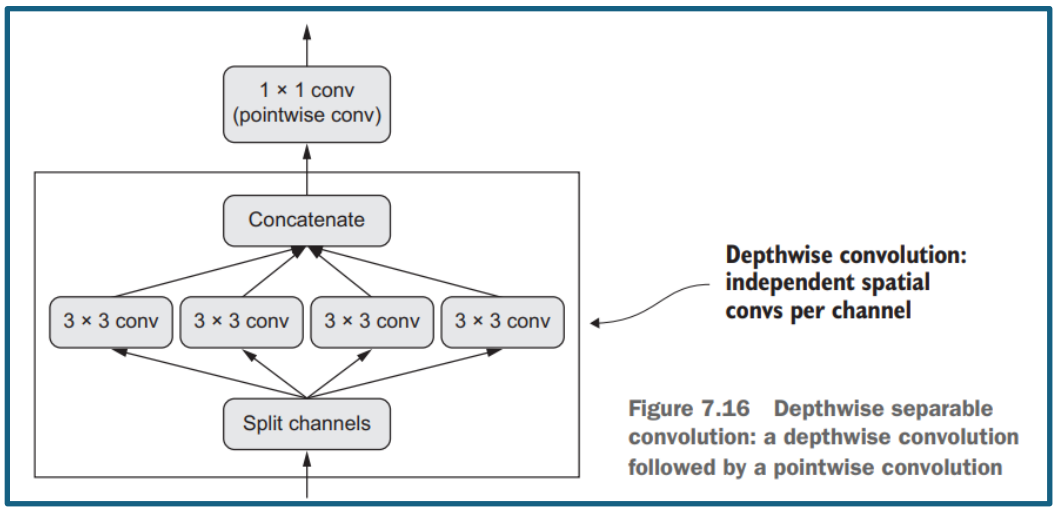

Figure 13 showcases an image from the Deep Learning with Python, Chapter 7 (Chollet, 2017a) regarding how depthwise convolutions function

The Depthwise Separable Convolution layer comprises a spatial convolution on each channel of its input independently, followed by a pointwise convolution to mix the output channels. This separation of spatial and channel-wise features is advantageous, especially when spatial locations in the input are highly correlated but different channels are fairly independent. By reducing the number of parameters and computations, Depthwise Separable Convolution leads to lighter and faster models with improved performance on classification tasks. Additionally, it enhances representation learning efficiency, allowing the model to learn better representations using less data. This approach aligns with the principles of the convolutional neural network design, as it results in models that are both computationally efficient and capable of achieving high accuracy.

As a result, the following is the fourth model architecture that was developed.

Layers	Parameters
resize_and_rescale	
data_augmentation	
SeparableConv2D	Filters: 32, kernel size: (3,3), activation = 'relu' input shape: (32, 128, 128, 3)
MaxPool2D	Pool size: (2,2)
SeparableConv2D	Filters: 64, kernel size: (3,3), activation = 'relu'
MaxPool2D	Pool size: (2,2)
SeparableConv2D	Filters: 64, kernel size: (3,3), activation = 'relu'
MaxPool2D	Pool size: (2,2)
Flatten	
Dense	Units: 64, activation = 'relu'
Dropout	0.5
Dense	Units: 64, activation = 'softmax'

Table 11: Information on the layers structure on model_V4

4.4.2 Results of model

Figure 14 shows the training and validation plots that the model achieved during training.

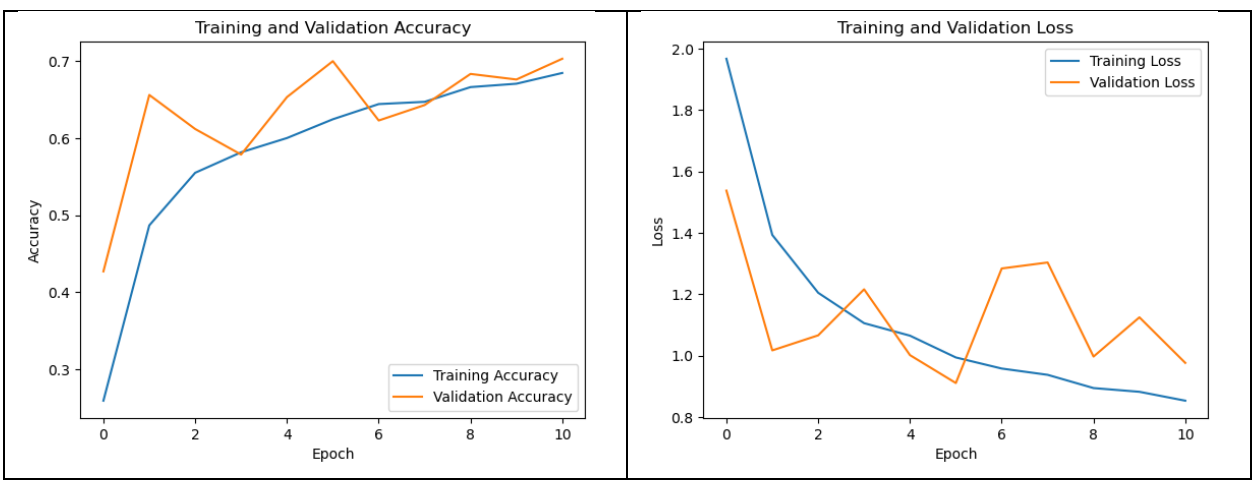

Figure 14 showing the training and validation plots for model_V4 on the test dataset

Similar to the model_V1 that was implemented, early stopping behavior observed during training. As depicted in the plot, the model training was halted at Epoch 11 out of 30, indicating that the model's performance did not improve significantly beyond this point. The maximum validation accuracy it achieved is 70.32%, far worse compared to the previous models.

Accuracy	0.690
Precision	0.716
Recall	0.690
F1	0.680

Table 12 shows the Evaluation metrics achieved by Model_V4 on the test dataset

Table 12 above shows the accuracy, precision, recall, and F1 score on the test set, with results hovering around 68.0% to 71.6%, which are much worse than model_V25 or other models that were implemented in this section. This could be due to the fact that SeparableConv2D layers are more computationally efficient and can learn representations using less data compared to traditional Conv2D layers, as a result be prone to overfitting. As a result, the early stopping mechanism was activated possibly leading to the training duration being insufficient for the model to converge and learn meaningful representations. Although the learning rate was also varied, it could also have been possible that it was too low, causing the optimization process to get stuck in a local minimum.

4.5.1 Utilization of model ensembling

In an attempt to create a model that could attain better results while staying fairly lightweight, model ensembling is utilized. Ensembling is a powerful technique in machine learning that combines multiple models to improve predictive performance. By leveraging the diverse perspectives and strengths of individual models, ensembling can effectively reduce variance, enhance generalization, and mitigate biases. Specifically, ensembling helps reduce variance by averaging or combining predictions from multiple models, leading to more robust predictions that are less sensitive to noise in the data. Additionally, each model in the ensemble may specialize in capturing different patterns or relationships within the data, allowing the ensemble to generalize better to unseen instances. Moreover, ensembling models with diverse architectures or training methodologies ensures that the ensemble captures a broader range of features and patterns, thereby improving overall predictive performance. I have chosen to employ model ensembling with model_V25 and model_V3_1, assigning them equal weightage(Figure 15).

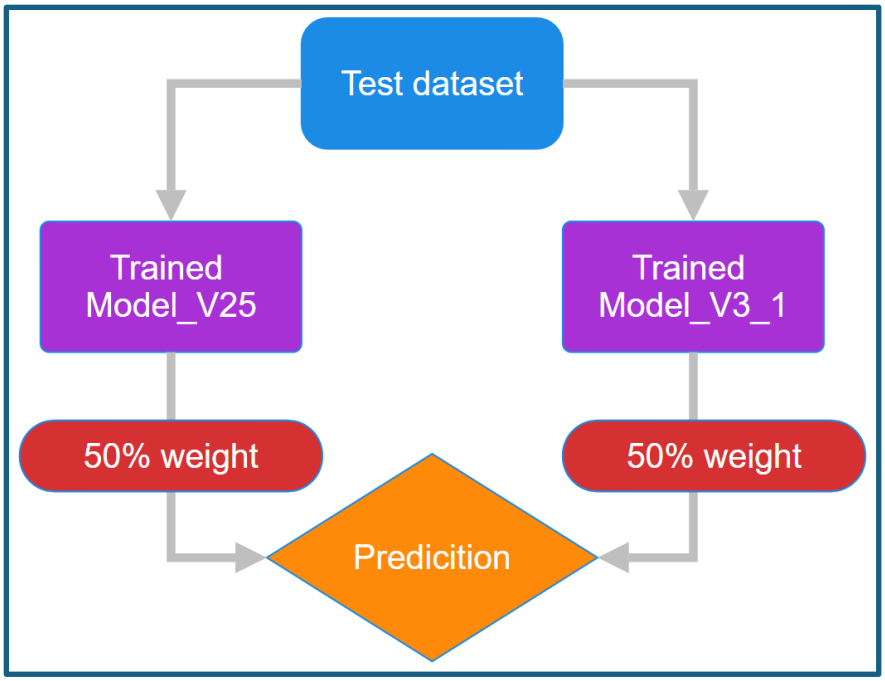

Figure 15 showcases the outline of the implementation of the ensemble model

In the case of model_V25 and model_V3_1, ensembling these models is motivated by the idea of leveraging their complementary strengths. Despite having similar overall performance metrics, their classification reports showcase different strengths and weaknesses. By combining these models, we aim to capitalize on their individual strengths while mitigating their weaknesses. For example, model_V25 achieves a high recall value of 95% while model_V3_1 achieves a lower score of 87% for class 4(Tomato Septoria leaf spot), where at the same time, it achieves a lower recall value of 87% for class 5 (Tomato Spider mites Two spotted spider mite) whereas model_V3_1 gets a comparatively higher score of 93%. Furthermore, since model_V25 and model_V3_1 have different architectures (e.g., different numbers of layers, different filters in each layer), ensembling them aligns with the principle of diversity in ensembling. By combining models with different architectures, we increase the likelihood of capturing a broader range of features and patterns from the data, ultimately improving the overall predictive performance of the ensemble model.

As the model architectures and training process have been discussed above, we can go straight into the results.

4.5.2 Results of model ensembling

Accuracy	0.966
Precision	0.966
Recall	0.966
F1	0.966

Table 13 shows the Evaluation metrics achieved by ensemble_model on the test dataset

Table 13 presents accuracy, precision, recall, and F1 scores for the ensemble_model on the test dataset, showing the highest performance metric achieved within this section, with values around 96.6% across all metrics.

	Precision	*Recall*	*F1*	*Support*
0	0.98	0.95	0.96	318
1	0.94	0.96	0.95	292
2	0.94	0.97	0.96	264
3	0.97	0.99	0.98	242
4	0.97	0.94	0.95	308
5	0.97	0.93	0.95	334
6	0.95	0.94	0.94	265
7	0.97	0.99	0.98	330
8	0.96	1.00	0.98	300
9	0.99	1.00	1.00	323

Table 14: Classification Report of ensemble_model on test dataset

The ensemble model demonstrates impressive performance across various metrics (Table 14), showcasing consistent or improved results for each class compared to individual models. Notably, it enhances or maintains precision, recall, and F1 score for both model_V3_1 and model_V25, attesting to its efficacy in capturing the strengths of each base model. While maintaining or enhancing overall performance, it's worth mentioning that there are isolated instances of slight detriments, such as a marginal decrease in precision for class 2 (Tomato Late Blight) from 95% in model_V25 to 94% in the ensemble model. Despite these minor trade-offs, the ensemble model excels in achieving a well-balanced classification performance across all classes, emphasizing its ability to combine the strengths of diverse models effectively.

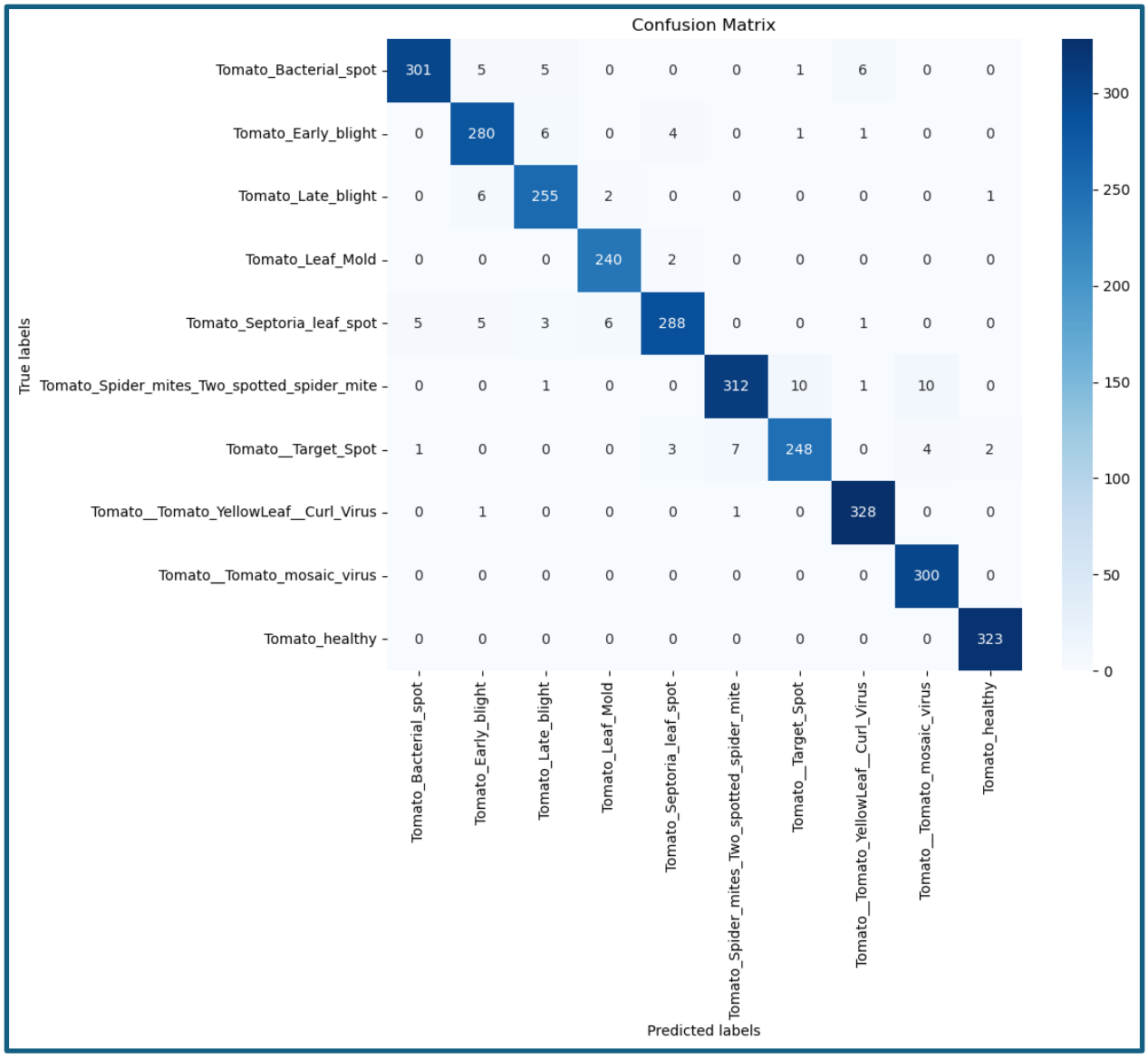

Figure 16 shows the confusion matrix of the final proposed ensemble_model

Figure 16 shows the true positive, false positive, true negative and false negative values attained for each class tested via the ensemble_model. For instance, in the last column, the model predicted 320 out of 323 healthy tomato leaves.

4.6 Examples of output from the final proposed model

The final model uses model ensembling of 2 iterated models developed – model_V25 and model_V3_1. The following figure shows the output of the final proposed model in the way described in the beginning of the project design.

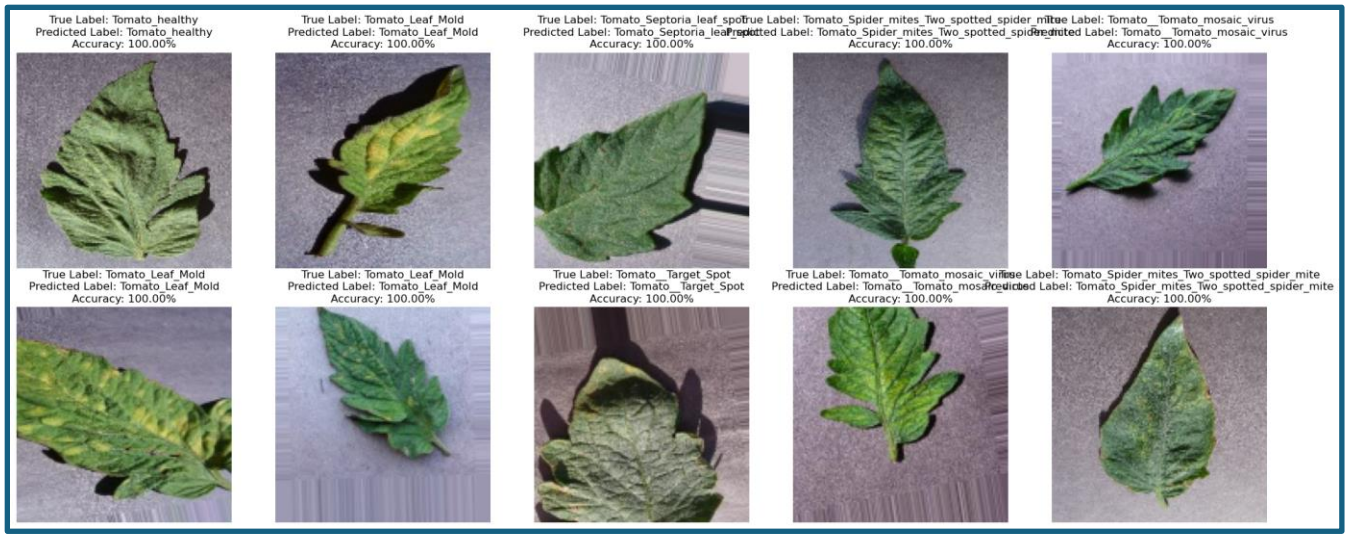

Figure 17 shows examples of images predicted from within the test set using the final proposed model

<u>Word Count of Chapter 4: 1994</u>

Chapter 5: Evaluation

In this section, a comprehensive evaluation will be conducted on all the models listed within this paper, and further comparison will be conducted with models established in the literature, to verify the effectiveness of the final ensemble model.

5.1 Overall evaluation of proposed models

The model development process began with an initial architecture comprising 2 sets of simple Conv2D and MaxPooling2D layers tailored for tomato leaf classification. This initial model served as a baseline for subsequent iterations and optimizations. Through iterative design, the architecture underwent significant modifications, with the further addition of layers along with regularization techniques such as dropout, and variations in filter sizes, addressing issues of overfitting. Further experimentation was conducted such as employing SeparableConv2D layers to optimize model complexity along with manual hyper-parameter tuning which aimed to enhance the performance metrics - accuracy, precision, recall, and F1 score. Through this series of experimentation and evolution, the best-performing solo model identified in chapter 4 was model_V25, as shown in Figure 18, displaying strong performance of around 95% for each metric.

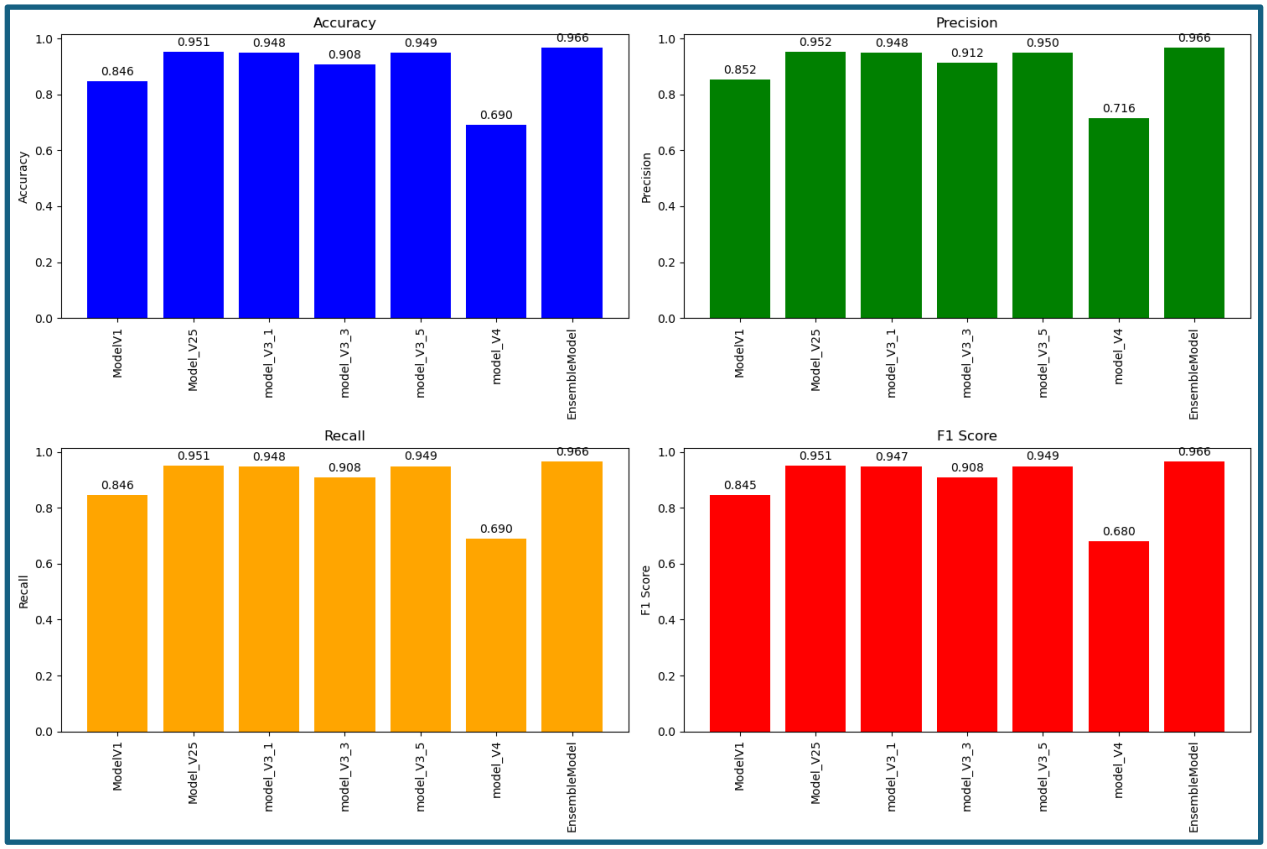

Figure 18 shows the metrics across all iterated models implemented in section 4

Subsequently, ensemble model development was utilized as an advanced strategy to further improve performance. By combining predictions from model_V25 and another model (model_V3_1) using equal weights, the ensemble model leveraged the complementary strengths of individual models, resulting in a more robust and accurate final prediction, achieving the best results of around 96.6%. The ensemble model's development marked the culmination of iterative design and evolution, demonstrating the effectiveness of combining diverse lightweight models to achieve superior performance in tomato leaf classification. Hence, the final proposed model architecture of this project can be attained from Table 5 and Table 8. The training parameters used to achieve the final results remain unchanged and can be attained from Table 3.

5.2 Performance comparison of the final proposed model with Traditional ML algorithms implemented

The figure below shows the comparison of the proposed model with implemented traditional ML models discussed in chapter 3. It is worth noting that various feature extraction techniques were implemented for the traditional models, such as Histogram of Oriented Gradients (HOG) and Local Binary Patterns (LBP). HOG focuses on capturing local intensity gradients in images, while LBP is a texture descriptor that identifies local patterns in grayscale images. Due to hardware limitations, these traditional models could only take in images of 64x64 resolution. Surprisingly, despite these preprocessing steps aimed at extracting higher-level features, the traditional ML models yielded worse results compared to using normalized pixel values of images sized 32x32 (downscaled due to hardware limitations), which was the main method employed for the generating the results seen in Figure 19.

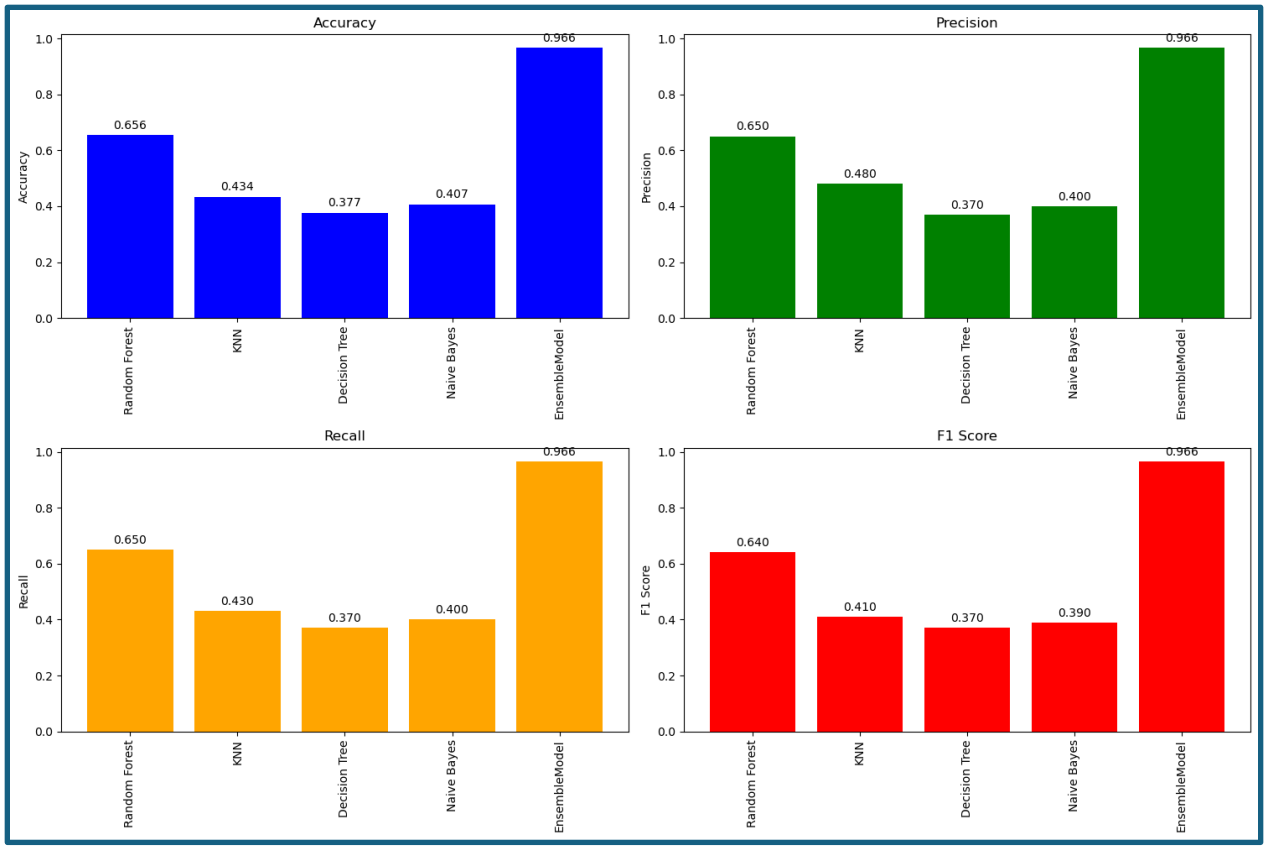

Figure 19 shows the chart of metrics comparing traditional models with proposed model

In the performance comparison of the final proposed model with traditional ML algorithms, the metrics across various models are starkly different. The proposed Ensemble Model significantly outperforms the traditional ML algorithms, including Random Forest, KNN, Decision Tree, and Naive Bayes, across all metrics of Accuracy, Precision, Recall, and F1 Score. For instance, while the Random Forest model achieves an accuracy of 65.6%, the Ensemble Model achieves a remarkably higher accuracy of 96.6%.

This could possibly be due to traditional models being constrained by their reliance on simpler representations and linear transformations, which may not adequately capture the diverse and intricate characteristics of real-world data. Unlike deep learning models, which can automatically learn and adapt their features and parameters through the iterative optimization process of gradient descent, traditional models are often limited by their fixed architectures and predefined feature sets. Furthermore, the fact that due to hardware limitation, the input image resolution (32x32) provided was a quarter of that (128x128), that was fed into the proposed deep model, could have resulted in the loss of fine-grained details and subtle patterns that could be crucial for accurate classification. This limitation, coupled with the inability of traditional models to effectively learn complex hierarchical representations, contributes to their inferior performance compared to deep learning models.

5.3 Performance comparison of the final proposed model with pre-trained models implemented

In the performance comparison (Figure 20) of the final proposed model with pre-trained models, several key observations can be made. Notably, the pre-trained models, such as MobileNet, EfficientNetB1, and Xception, achieve remarkably high scores across all metrics, including Accuracy, Precision, Recall, and F1 Score, with values consistently close to 99.3%. These models demonstrate exceptional performance in accurately classifying the images, and are closely followed by ResNet50, which performs on average of 97.6%. Our ensemble model ranks after these pre-trained models showcasing competitive scores of 96.6%. On the other hand, models like the VGG19 and MobileNetV2 surprisingly yielded slightly lower scores across the metrics compared to the aforementioned models but still maintain respectable performance levels.

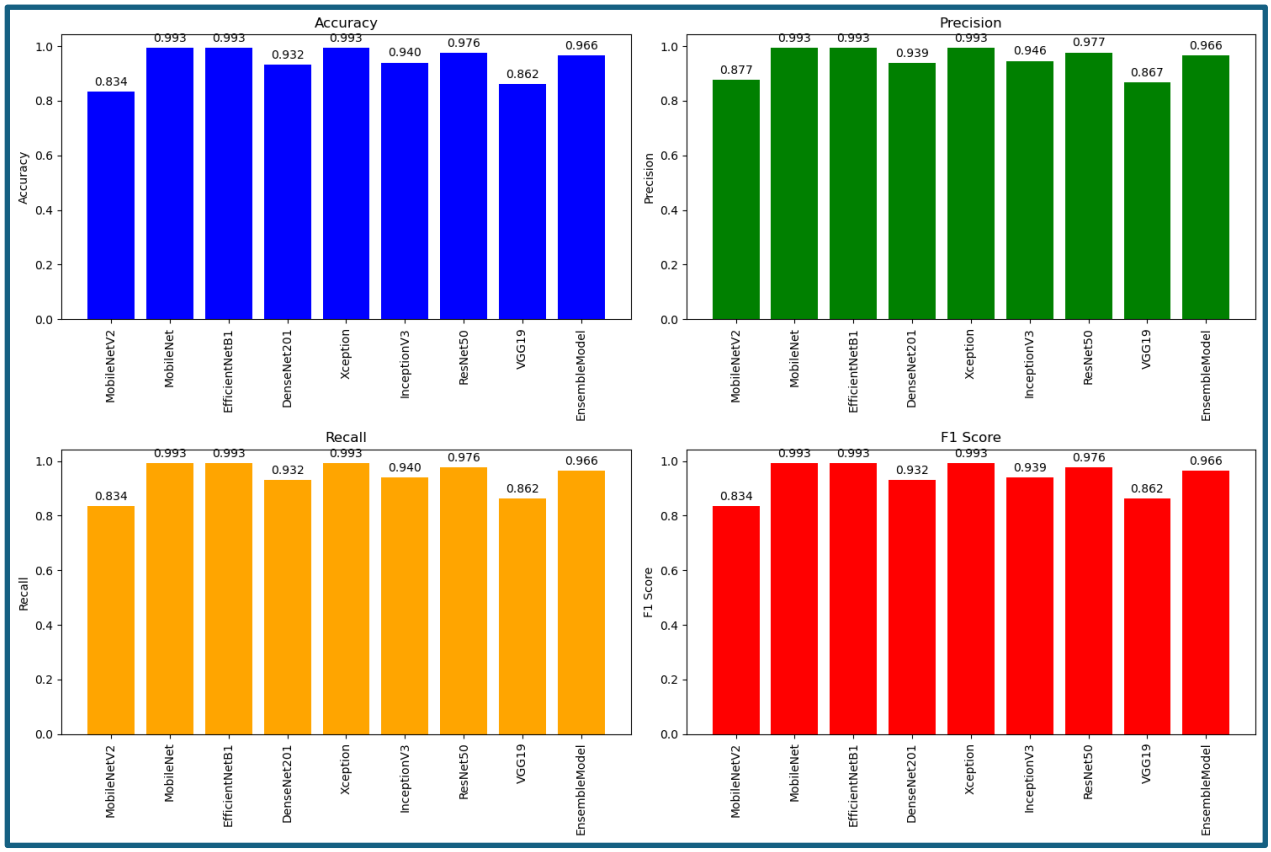

Figure 20 shows the charts of metrics comparing pre-trained models with proposed model

VGG19 and MobileNetV2 may not have performed as well as other pre-trained models due to differences in their architectures and capabilities for transfer learning. VGG19, with its deeper architecture comprising multiple convolutional layers, may have faced challenges in adapting to the specific features and patterns present in the dataset, leading to less effective transfer learning. Similarly, MobileNetV2, although designed for efficiency and speed, may have lacked the depth

and complexity necessary to capture intricate patterns in the dataset, resulting in suboptimal performance during transfer learning.

5.4 Complexity Comparison between Models

The following table showcases a holistic comparison of accuracy and complexity between all the models that were implemented within the paper and are sorted via depth. The parameters, depth and size for the pre-trained models were retrieved from Keras Applications website[7], while for the remaining models, it was calculated during implementation.

Models	Accuracy	Model Size(MB)	Parameters	Depth
Random Forest	0.656	128	-	-
KNN	0.434	549	-	-
Decision Tree	0.377	1	-	-
Naïve Bayes	0.407	0.47	-	-
Model_V1	0.846	2.35	614,730	5
Model_V25	0.951	3.28	859,850	9
Model_V4	0.690	3.09	810,821	9
Model_V3_1	0.948	1.57	412,618	13
Model_V3_3	0.908	1.54	403,898	13
Model_V3_5	0.949	0.64	166,602	13
VGG19	0.862	549	143,700,000	19
Ensemble Proposed[8]*	0.966	3.28+1.57	412,618 + 859,850	9 + 13
MobileNet	0.993	16	4,300,000	55
Xception	0.993	88	22,900,000	81
MobileNetV2	0.834	14	3,500,000	105
ResNet50	0.976	98	25,600,000	107
EfficientNetB1	0.993	31	7,900,000	186
InceptionV3	0.940	92	23,900,000	189
DenseNet201	0.932	80	20,200,000	402

Table 15 showing the complexity information of models implemented in this paper, along with the accuracy

In the complexity comparison between models (Table 15), it's evident that the depth of the model architecture doesn't always correlate with better results. For instance, among the iterated models implemented in this report, Model_V25 stands out with a depth of 8 yet achieves superior

[7] https://keras.io/api/applications/

[8] The details of the proposed model are simply attained via adding that of model_V25 and model_V3_1.

accuracy compared to models with a depth of 13, such as Model_V3_1 and Model_V3_3. This discrepancy accentuates the importance of model design and optimization beyond simply increasing depth. Furthermore, the ensemble proposed model, which technically combines models with depths of 9 and 13, outperforms complex pre-trained models like InceptionV3, DenseNet201, and ResNet50, despite their significantly higher depths. This observation highlights the effectiveness of ensemble techniques and demonstrates the successful achievement of the project's objective in developing a robust and accurate classification model.

The evaluation based on the size of the models reveals an intriguing observation: despite being significantly smaller than established models like VGG19, MobileNet, and Xception, the models implemented within this paper demonstrate competitive performance, even surpassing some of the larger models in terms of accuracy. This phenomenon suggests that there may exist a 'sweet spot' within model size, complexity, and performance that is often understated in the field. This 'sweet spot' implies that models don't necessarily need to be excessively large or complex to achieve high accuracy, and more efforts could be focused on unraveling the intricate relationships between model size, complexity, and performance to optimize model design and development further.

5.5 Comparison of Proposed Ensemble Model with those in existing literature

Table 16 depicts the comparison with existing literature, showcasing the competitive performance of the proposed ensemble model when compared to Durmus's AlexNet and SqueezeNet, based on accuracy and size. The proposed model achieves an accuracy of 0.966 with a size of 4.85 MB, outperforming Durmus's AlexNet (accuracy: 0.957, size: 228 MB) and SqueezeNet (accuracy: 0.943, size: 2.9 MB).

It's worth noting that the difference in the results of the pre-trained models between this study and Chowdhury's could be attributed to the difference in frameworks, with Chowdhury using PyTorch while this study utilizes Keras. Furthermore, the discrepancy in training epochs and loss functions also plays a role, as this study employs 10 epochs with sparse categorical cross-entropy, while Chowdhury's study uses 15 epochs with binary cross-entropy. Despite these variations, the proposed ensemble model demonstrates competitive results overall, showcasing the potential for achieving high accuracy with a fraction of the size.

Model	Paper	Accuracy	Size(MB)
AlexNet	(Durmuş et al., 2017)	0.957	228
SqueezeNet	(Durmuş et al., 2017)	0.943	2.9
Proposed Model	This study	0.966	4.85
MobileNet	(Chowdhury et al., 2021)	0.972	-
MobileNet	This study	0.993	16
DenseNet201	(Chowdhury et al., 2021)	0.980	-
DenseNet201	This study	0.932	80
InceptionV3	(Chowdhury et al., 2021)	0.974	-
InceptionV3	This study	0.940	92

Table 16: Accuracy comparison with other models from literature review

The deployment of models in the field for tasks like tomato disease leaf classification requires considerations beyond just accuracy. Models like the proposed ensemble model, with their smaller size, are particularly advantageous for deployment in resource-constrained environments such as agricultural settings. The smaller size of these models enables them to be deployed on mobile devices, IoT platforms or even edge devices with limited computational resources, allowing for real-time inference directly on the field. Furthermore, the ensemble model's adaptability to different environmental conditions and variations in leaf images enhances its versatility and applicability across various geographical regions and cultivation practices. Overall, models like the proposed ensemble model represent a significant advancement in the field of agricultural technology, offering efficient and accurate solutions for tomato disease leaf classification and contributing to sustainable crop production.

Word Count of Chapter 5: 1387

Chapter 6: Conclusion

In this paper, after conducting a series of iterations and experimentations on a basic CNN structure, we proposed an ensemble model to address the challenge of tomato leaf disease classification. Leveraging the PlantVillage dataset from Kaggle, our approach required data augmentation for class balancing and involved the implementation of traditional machine learning algorithms and pre-trained models for comprehensive performance evaluation. The results showcase competitive performance, achieving an accuracy of 96.6%, despite the model's compact size compared to more complex counterparts. This highlights the effectiveness of our ensemble CNN approach in achieving accurate and efficient disease classification for tomato plants. As a further work in this field, more data could be collected showcasing the different stages of progression of disease, and possibly even likelihood of recovery. Greater data availability could prompt the modification of the proposed model or even the search for better, efficient architectures.

On a broader scale, there's a growing recognition that, rather than large abstract models with relatively high general accuracy, more focus should be directed towards small, specific models tailored to particular use cases. These smaller models, when appropriately designed and trained, can offer comparable or even superior performance while being more resource-efficient in terms of computational requirements and memory footprint. Additionally, exploring the concept of ensembling smaller models can further enhance the robustness and reliability of the deployed systems, leveraging the strengths of individual models while mitigating their weaknesses. This approach represents an exciting avenue for research and development, aiming to discover the 'sweet spot' where model size, complexity, and performance converge to maximize efficiency and effectiveness in real-world deployment scenarios.

Word Count of Chapter 6: 260 words

Overall Word Count: 7922

Reference

Ahmad, A., Saraswat, D., & El Gamal, A. (2023). A survey on using deep learning techniques for plant disease diagnosis and recommendations for development of appropriate tools. *Smart Agricultural Technology*, *3*, 100083. https://doi.org/10.1016/j.atech.2022.100083

Ashok, S., Kishore, G., Rajesh, V., Suchitra, S., Sophia, S. G. G., & Pavithra, B. (2020). Tomato Leaf Disease Detection Using Deep Learning Techniques. *2020 5th International Conference on Communication and Electronics Systems (ICCES)*, 979–983. https://doi.org/10.1109/ICCES48766.2020.9137986

Bock, C. H., Poole, G. H., Parker, P. E., & Gottwald, T. R. (2010). Plant Disease Severity Estimated Visually, by Digital Photography and Image Analysis, and by Hyperspectral Imaging. *Critical Reviews in Plant Sciences*, *29*(2), 59–107. https://doi.org/10.1080/07352681003617285

Chollet, F. (2017a). *Deep Learning with Python* (2nd ed.). Manning. https://www.manning.com/books/deep-learning-with-python

Chollet, F. (2017b). *Xception: Deep Learning with Depthwise Separable Convolutions* (arXiv:1610.02357). arXiv. https://doi.org/10.48550/arXiv.1610.02357

Chowdhury, M. E. H., Rahman, T., Khandakar, A., Ibtehaz, N., Khan, A. U., Khan, M. S., Al-Emadi, N., Reaz, M. B. I., Islam, M. T., Ali, S. H. M., Chowdhury, M. E. H., Rahman, T., Khandakar, A., Ibtehaz, N., Khan, A. U., Khan, M. S., Al-Emadi, N., Reaz, M. B. I., Islam, M. T., & Ali, S. H. M. (2021). Tomato Leaf Diseases Detection Using Deep Learning Technique. In *Technology in Agriculture*. IntechOpen. https://doi.org/10.5772/intechopen.97319

Devaraj, A., Rathan, K., Jaahnavi, S., & Indira, K. (2019). Identification of Plant Disease using Image Processing Technique. *2019 International Conference on Communication and Signal Processing (ICCSP)*, 0749–0753. https://doi.org/10.1109/ICCSP.2019.8698056

Durmuş, H., Güneş, E. O., & Kırcı, M. (2017). Disease detection on the leaves of the tomato plants by using deep learning. *2017 6th International Conference on Agro-Geoinformatics*, 1–5. https://doi.org/10.1109/Agro-Geoinformatics.2017.8047016

FAO - News Article: Climate change fans spread of pests and threatens plants and crops, new FAO study. (2021, February 6). https://www.fao.org/news/story/en/item/1402920/icode/

Frusciante, L., Carli, P., Ercolano, M. R., Pernice, R., Di Matteo, A., Fogliano, V., & Pellegrini, N. (2007). Antioxidant nutritional quality of tomato. *Molecular Nutrition & Food Research*, *51*(5), 609–617. https://doi.org/10.1002/mnfr.200600158

Fuentes, A., Yoon, S., Kim, S. C., & Park, D. S. (2017). A Robust Deep-Learning-Based Detector for Real-Time Tomato Plant Diseases and Pests Recognition. *Sensors*, *17*(9), Article 9. https://doi.org/10.3390/s17092022

Global Alliance for Improved Nutrition (GAIN). (2020). *Tomatoes—The world's most popular vegetable*. GAIN. https://www.gainhealth.org/resources/reports-and-publications/tomatoes-worlds-most-popular-vegetable

He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition* (arXiv:1512.03385). arXiv. https://doi.org/10.48550/arXiv.1512.03385

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications* (arXiv:1704.04861). arXiv. https://doi.org/10.48550/arXiv.1704.04861

Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2018). *Densely Connected Convolutional Networks* (arXiv:1608.06993). arXiv. https://doi.org/10.48550/arXiv.1608.06993

Hughes, D. P., & Salathe, M. (2015, November 25). *Papers with Code—An open access repository of images on plant health to enable the development of mobile disease diagnostics*. https://cs.paperswithcode.com/paper/an-open-access-repository-of-images-on-plant

Klein, B. (2024, March 6). *8. K-Nearest Neighbor Classifier in Python | Machine Learning*. Python-Course.Eu. https://python-course.eu/machine-learning/k-nearest-neighbor-classifier-in-python.php

Mahlein, A.-K. (2016). Plant Disease Detection by Imaging Sensors – Parallels and Specific Demands for Precision Agriculture and Plant Phenotyping. *Plant Disease*, *100*(2), 241–251. https://doi.org/10.1094/PDIS-03-15-0340-FE

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, *12*(85), 2825–2830.

Ram, R., Molla, K., Govindharaj, G. P. P., S. R, P., Baite, M., Yadav, M., & Jena, M. (2018). *Recent Trends in Detection and Diagnosis of Plant Diseases* (pp. 25–46).

Rani, F. A., Kumar, S. N., Haridhas, A. K., Dyson, C., Varuvel, S., & Jeba, P. S. (2019). *K-means Clustering and SVM for Plant Leaf Disease Detection and Classification*. 1–4. https://doi.org/10.1109/ICRAECC43874.2019.8995157

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2019). *MobileNetV2: Inverted Residuals and Linear Bottlenecks* (arXiv:1801.04381). arXiv. https://doi.org/10.48550/arXiv.1801.04381

Simonyan, K., & Zisserman, A. (2015). *Very Deep Convolutional Networks for Large-Scale Image Recognition* (arXiv:1409.1556). arXiv. https://doi.org/10.48550/arXiv.1409.1556

Sladojevic, S., Arsenovic, M., Anderla, A., & Stefanović, D. (2016). Deep Neural Networks Based Recognition of Plant Diseases by Leaf Image Classification. *Computational Intelligence and Neuroscience*, *2016*, 1–11. https://doi.org/10.1155/2016/3289801

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2015). *Rethinking the Inception Architecture for Computer Vision* (arXiv:1512.00567). arXiv. https://doi.org/10.48550/arXiv.1512.00567

Tan, M., & Le, Q. V. (2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* (arXiv:1905.11946). arXiv. https://doi.org/10.48550/arXiv.1905.11946

Appendix

The code for the Tomato Disease Leaf Detection project can be found in the GitHub repository at the following link: Tomato Disease Leaf Detection Repository.